

Detailní technický popis · zabezpečení

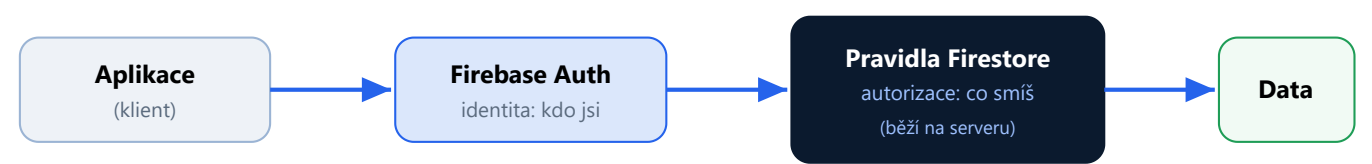
TORKIS

Bezpečnostní pravidla Firestore — jak fungují

Verze aplikace 5.1.8 · Stav k 4. 6. 2026 · Projekt `visto-51cb7` · Ověřeno 32 automatickými testy

1. Princip: kde a kdy se pravidla vyhodnocují

Bezpečnostní pravidla Firestore jsou kód, který běží na **serverech Google** u **každého** přístupu k databázi. Nejsou součástí aplikace — proto je **nelze obejít** úpravou klienta nebo přímým připojením k databázi.



Obr. 1 — Každý požadavek projde přes server: nejdřív identita (Auth), pak autorizace (pravidla), teprve pak data.

Tři základní zásady našich pravidel:

- **Vše zakázáno, dokud to pravidlo výslovně nepovolí** (deny by default).
- **Izolace servisů** — každý vidí jen data svého `servis_id` .
- **Čtení = jednotlivý dokument (get) i dotaz (list)** — u dotazu navíc platí zvláštní podmínka (viz kap. 5).

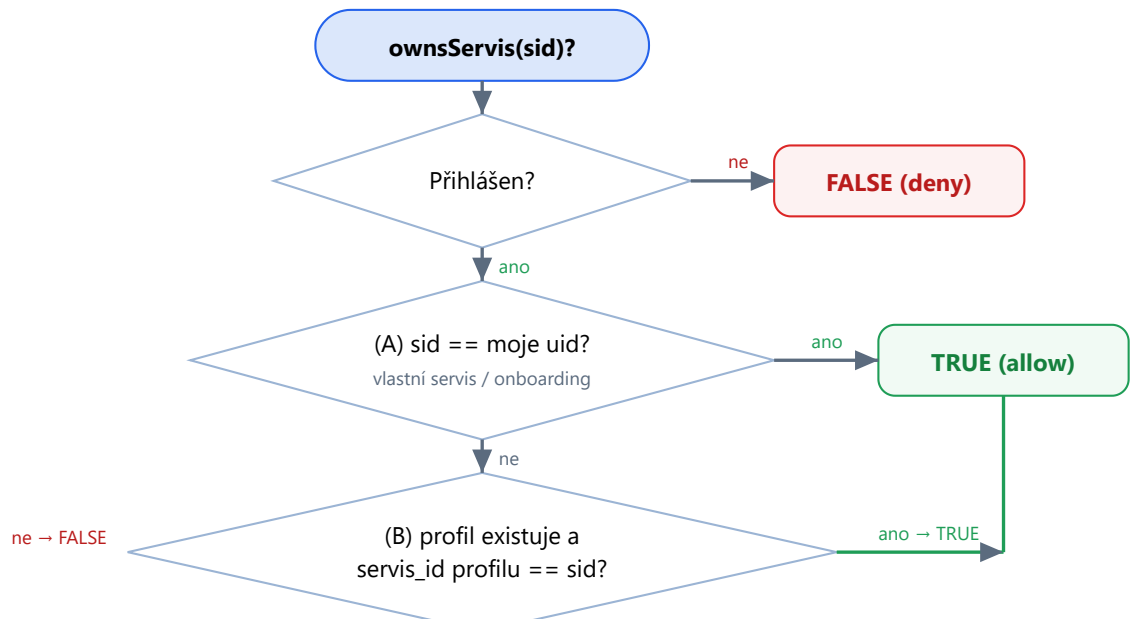
2. Pomocné funkce

Pravidla stojí na několika funkcích, které se volají opakovaně:

Funkce	Vrací true, když...
<code>signedIn()</code>	uživatel je přihlášený a není anonymní (mechanik/majitel).
<code>isAnonymous()</code>	jde o anonymní relaci zákaznického portálu.
<code>isOwner()</code>	e-mail uživatele je e-mail vlastníka aplikace (správa předplatného).
<code>myServisId()</code>	(pomocné) přečte <code>servis_id</code> z profilu volajícího v kolekci <code>uzivatele</code> .
<code>ownsServis(sid)</code>	daný <code>servis_id</code> patří volajícímu — viz rozhodovací strom níže.
<code>canManageStaff(sid)</code>	volající je člen servisu a má právo <code>prava.zamestnanci</code> .

Klíčová funkce `ownsServis()` — anotovaný kód

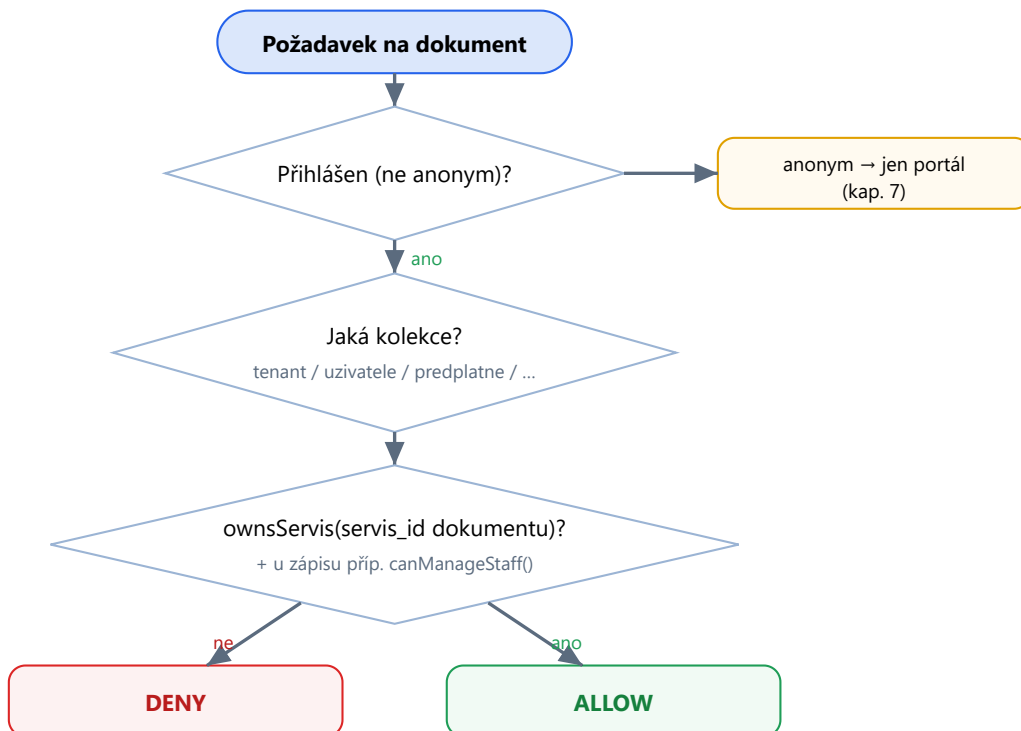
```
// Patří daný servis_id přihlášenému uživateli?
function ownsServis(sid) {
  return signedIn() && (
    sid == request.auth.uid // (A) bootstrap / vlastní servis
    || (exists(.../uzivatele/${request.auth.uid}) // (B) profil existuje
        && myServisId() == sid) // a jeho servis_id sedí
  );
}
```



Obr. 2 — Rozhodování `ownsServis()`. Větev (A) je důležitá pro onboarding (profil ještě neexistuje), větev (B) pro zaměstnance.

3. Rozhodovací tok autorizace

Když přijde požadavek na čtení/zápis dokumentu, server projde tento tok:



Obr. 3 — Zjednodušený tok pro datové (tenantní) kolekce. Konkrétní kolekce mají dílčí pravidla z kap. 4.

4. Mapa kolekcí → pravidla

Kolekce	Klíč	Čtení	Zápis
vozidla, zakaznici, ukony, faktury, vin_skeny, value_skeny	pole servis_id	ownsServis	ownsServis
zakazky	pole servis_id	ownsServis nebo anonym + portal_token	ownsServis
nastaveni_servisu	ID = servis_id	ownsServis(id)	ownsServis(id)
uzivatele	ID = uid	vlastní nebo kolega (ownsServis)	self (zamčené pole) / canManageStaff
predplatne	ID = servis_id	ownsServis / owner	owner / jen trial
znacka	—	přihlášený	owner
maily	—	—	jen vytvořit
chybove_logy	—	owner	jen vytvořit
pouziti, vin_cache, value_cache	—	X (jen backend)	X (jen backend)

člen servisu

anonymní portál

jen backend / owner

5. Dotazy (list) — proč musí klient filtrovat

U dotazu, který vrací více dokumentů, pravidla **nefiltrují** výsledek — buď celý dotaz **povolí**, nebo **odmítnou**. Firestore povolí dotaz jen tehdy, je-li **zaručeno**, že vrátí pouze dokumenty splňující pravidlo. Proto musí klient sám přidat filtr `where('servis_id' == můj_servis)` — jinak je dotaz zamítnut.

✓ Správně



✗ Bez filtru / cizí servis

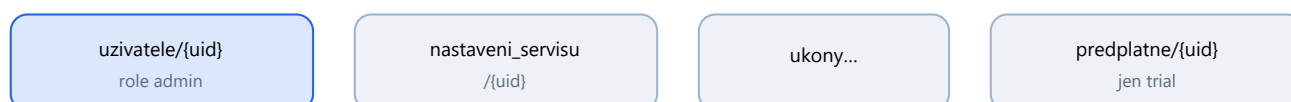


Obr. 4 — Aplikace TORKIS u všech seznamů filtruje podle `servis_id`, takže dotazy projdou; útočníkův „dej mi vše“ dotaz server odmítne.

6. Onboarding — proč existuje větev „sid == uid“

Při prvním nastavení se v **jednom dávkovém zápisu** zakládá profil uživatele i nastavení servisu, úkony a trial. V té chvíli ale profil `uzivatele` ještě **není uložen**, takže by se z něj nedalo přečíst `servis_id`. Řeší to větev (A) v `ownsServis()`: zakladatel zakládá svůj servis pod **vlastním uid** (`servis_id == uid`), což pravidlo pustí i bez existujícího profilu.

Dávkový zápis při onboarding (servis_id = uid zakladatele)

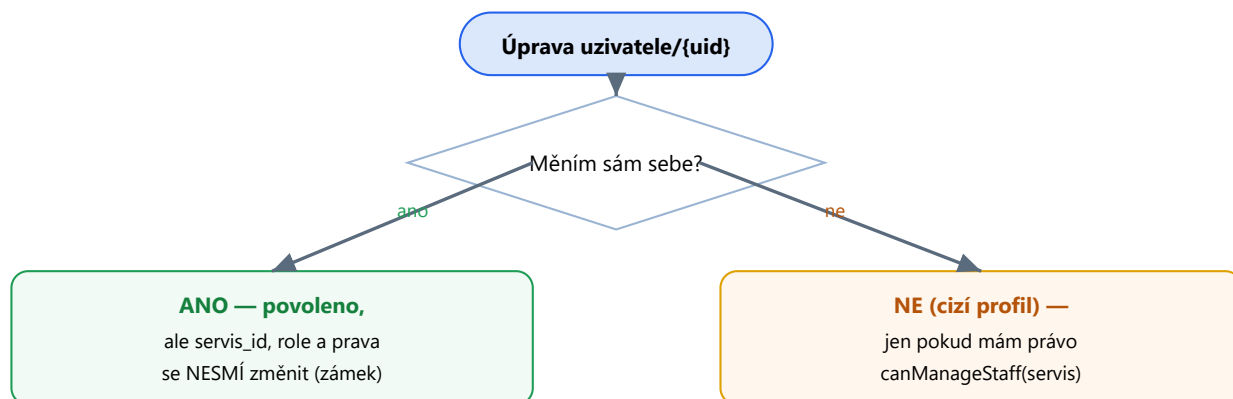


vše projde přes větev (A): `servis_id == request.auth.uid` — profil zatím nemusí existovat

Obr. 5 — Bootstrap onboarding: jeden dávkový zápis, povolený díky shodě `servis_id` s `uid` zakladatele.

7. Profil uživatele: vlastní úpravy vs. správa, blok eskalace

Pravidlo pro úpravu profilu rozlišuje, zda uživatel mění **sebe**, nebo **někoho jiného**:

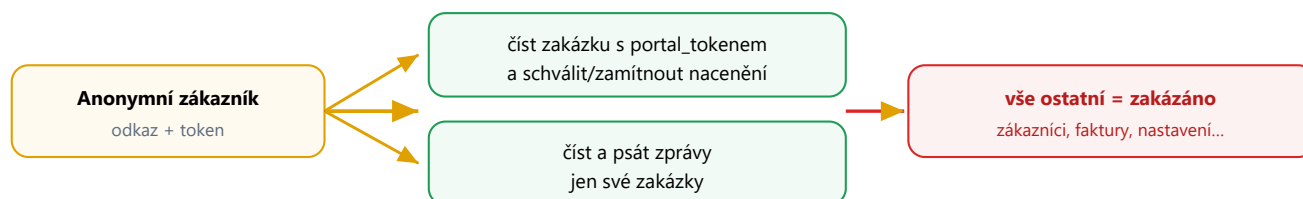


Obr. 6 — Self-update nesmí měnit *role* / *servis_id* / *práva* → uživatel se nemůže sám povýšit ani přeskóčit do cizího servisu.

Zablokovaný útok: zaměstnanec by si chtěl nastavit *role: admin* nebo *servis_id* jiného servisu. Pravidlo to odmítne — pole jsou při úpravě vlastního profilu zamčená a správu cizích profilů má jen pověřený správce.

8. Zákaznický portál (anonymní přístup)

Zákazník otevře svou zakázku přes odkaz s jednorázovým tokenem. Přihlásí se **anonymně** a pravidla mu povolí jen:



Obr. 7 — Anonymní portál má extrémně úzký přístup, omezený na jednu zakázku a její zprávy.

9. Serverové kolekce a „vše ostatní zakázáno“

Kolekce, které drží limity placených funkcí — počítadlo využití (*pouziti*) a cache (*vin_cache* , *value_cache*) — klient **nesmí číst ani zapisovat**. Mění je jen backend (Cloud Functions přes Admin SDK, který pravidla obchází, ale je dostupný jen serveru). Cokoli, co není ve výše uvedených pravidlech, je **ve výchozím stavu zamítnuto** — nová kolekce tedy není omylem otevřená.

10. Ověřování pravidel (testy)

Pravidla jsou pokryta **32 automatickými testy** v lokálním Firestore emulátoru (izolace servisů, dotazy, onboarding, eskalace, serverové kolekce, portál). Postup při změně: upravit pravidla → spustit testy v emulátoru → teprve pak nasadit (`firebase deploy --only firestore:rules`). Testy jsou ve složce `tools/rules_test/`.

Shrnutí: Pravidla běží na serveru, ve výchozím stavu vše zakazují a otevírají přístup výhradně členům správného servisu. Onboarding je ošetřen speciální bootstrap větví, samostatně je ošetřena správa

uživatelů (s blokem eskalace práv), zákaznický portál i serverové kolekce. Vše je ověřeno automatickými testy.

TORKIS — detailní popis bezpečnostních pravidel Firestore. Vygenerováno 4. 6. 2026 (verze aplikace 5.1.8). Diagramy jsou ilustrativní; závazný je vždy aktuální soubor `firestore.rules`.